CASE STUDY 131: Spice Grinding Unit Order Fulfilment *Primary syllabus focus: Transactions, ACID and Concurrency*

1. Title

Spice Grinding Unit Order Fulfilment Database Using SQL

2. Problem Statement

A spice grinding unit takes bulk orders for ground spice blends, produces them in batches against a recipe, and packs them for dispatch to retailers.

The database must support the day to day working of the unit manager and produce the reporting needed at the close of every production day.

blends is the master list of every spice blend the unit manager deals with — concrete examples would be things like Garam Masala, Chilli Powder or Turmeric Powder — and each one carries its price per kg (in ₹), the figure everything else in the system is measured against. dispatches is the day to day activity log: one row every time a dispatch actually happens, carrying its own kg dispatched recorded against a dispatch date, and always pointing back to exactly one spice blend through blend_id. Nothing in dispatches should ever exist without a matching row in blends.

3. Objectives

A complete solution for this case needs four tables working together. blends and dispatches were introduced above; you also need to design, on your own, something like the two tables below — production_batches and orders — so every rupee/record in the final reporting can be traced all the way back to where it started:

production_batches

production_batches is the grinding floor's output record: one row per batch of a blend actually ground and packed, with the quantity produced, so a dispatch can only be matched against a batch that genuinely exists and is complete.

id	blend_id	batch_date	kg_produced
1	1	2024-01-12	500
2	2	2024-01-20	350

3	1	2024-02-05	620

orders

orders is the customer demand record: one row per quantity of a blend a customer has ordered, so a dispatch can be checked against what was actually ordered instead of shipping out whatever quantity happens to be on hand.

id customer_name blend_id kg_ordered order_date

1 Sri Ram Traders 1 300 2024-01-10

2	Annapurna Condiments	2	200	2024-01-18
3	Sri Ram Traders	1	400	2024-02-01

Students should:

- Model recipes, raw spice stock, production batches, orders and dispatches so a dispatched order always matches a completed batch.

Sample Data for This Case Study

This is a small, deliberately raw preview of the two central tables — blends and dispatches — so you know exactly what kind of columns and values your own, larger seeded database needs to carry before you design it.

blends

id	name	city	price_per_kg_rs
1	Garam Masala	Jaipur	1005
2	Chilli Powder	Ahmedabad	980
3	Turmeric Powder	Chennai	455
4	Sambar Masala	Jaipur	1080
5	Kitchen King Masala	Bengaluru	555
6	Pav Bhaji Masala	Ahmedabad	580

Database Management Systems

dispatches

id	blend_id	kg_dispatched	dispatch_date
1	1	239	2024-01-08
2	1	248	2024-01-19
3	2	239	2024-02-02
4	3	194	2024-02-14
5	3	59	2024-02-27
6	4	59	2024-03-05
7	5	266	2024-03-16
8	5	149	2024-03-28

Using this sample data, write queries that:

- Write a transaction (BEGIN ... COMMIT) that inserts one new dispatch, and explain what a ROLLBACK at that point would undo.
- Describe, using this schema, what a lost update would look like if two staff members updated the same dispatch's kg_dispatched at the same time.
- Write the SQL for a SAVEPOINT inside a multi-step transaction, and describe one step you would roll back to it.

- Pick an isolation level for recording a new dispatch as a business operation here, and justify it in two lines.
- Write a transaction that inserts a new spice blend and its first dispatch as a single atomic unit, and explain what should happen if the second insert fails.
- If you kept a derived 'is_active' flag on blends, explain why it would need a trigger to stay correct.

Then extend this exact table shape into a realistic full-scale blends and dispatches dataset for your submission (tens of rows, not six), and re-run the same style of queries against it — filtering, a join, a GROUP BY/HAVING report and at least one nested or correlated subquery — recording why each design decision was made.

4. Outcomes

On completing this case study, the student:

- Explains each ACID property using evidence from their own database rather than a definition.
- Reproduces a lost update or dirty read scenario and describes the fix that closes it.
- Justifies the isolation level chosen for a given business operation.
- Defends this design against a reasonable alternative instead of asserting that it is correct.
- Explains why each query was written the way it was, and what it would cost to write it differently.

5. Deliverables

- One project.sql file with your schema, seed data and every query you write for your own blends and dispatches tables — a comment with your name above each query, and the query itself written directly below that comment.
- One PDF report with your ER diagram and one screenshot per query, run on your own terminal. Each screenshot must show, in this order: a comment with your name and roll number (for example --Name - Chandrika Roll No - 101), the query itself, and its result — all captured together in that one screenshot, in the same order as the .sql file.